

Absolutely. From now on, **one topic per answer only**. I'll follow the uploaded lecture in its exact order and explain it like a teacher, without jumping ahead.

# Lecture 2 — Introduction to Problem Solving

## Topic 1: What is a Problem?

### 1. Basic idea

In computer science, a **problem** is something that needs to be solved.

More formally:

A problem is a request where we transform some **inputs** into **outputs** according to specific rules and constraints.

Think of it as:

**What I have** → **Problem-solving process** → **What I want**

For example:

**I have:** marks of a student

**I want:** the student's grade

The computer needs rules to transform the marks into the correct grade.

---

## 2. The 5 important parts of a problem

Your lecture gives us five things to identify:

### ① Input

**Input = What the program receives.**

Examples:

- Numbers
- Names
- Marks
- Passwords
- Prices

- Text

Example:

Enter your marks: 85

Here, 85 is the **input**.

---

## ② Process

**Process = What we do with the input.**

For example, suppose we receive marks:

85

We might apply this rule:

If marks  $\geq 80 \rightarrow$  Grade A

That decision is the **process**.

The process can contain:

- Calculations
  - Formulas
  - Decisions
  - Comparisons
  - Rules
  - Thresholds
- 

## ③ Output

**Output = What the program produces.**

For our example:

Grade A

So:

Input  $\rightarrow$  85

Process  $\rightarrow$  Check marks

Output  $\rightarrow$  Grade A

---

## ④ Constraints

**Constraints = Restrictions or limits on the problem.**

For example:

Enter marks between 0 and 100.

Then:

$$0 \leq \text{marks} \leq 100$$

A mark of 85 is valid.

A mark of 150 is not valid.

Constraints can involve:

- Valid ranges
  - Time limits
  - Memory limits
  - Required formats
  - Maximum/minimum values
- 

## ⑤ Edge Cases

**Edge cases = unusual, boundary, or tricky situations.**

For example, if we're calculating an average:

marks = 0

What happens?

Or if we're dividing two numbers:

10 / 0

That's a special case because division by zero isn't allowed.

Other examples:

- Empty input
- Zero values
- Minimum value
- Maximum value
- Equal values
- Invalid input

Your lecture specifically identifies edge cases as rare or tricky situations that need attention.

---

### 3. The IPOCE model

A very important thing to remember from this lecture is:

**I — P — O — C — E**

| Letter | Meaning     | Question                           |
|--------|-------------|------------------------------------|
| I      | Input       | What comes in?                     |
| P      | Process     | What do we do with it?             |
| O      | Output      | What comes out?                    |
| C      | Constraints | What limits/restrictions exist?    |
| E      | Edge Cases  | What unusual situations can occur? |

The lecture calls this the “**I-P-O-C-E**” **checklist** for quickly understanding a problem.

---

### 4. Real-life example

Suppose someone says:

"Calculate the total price of 3 shirts if each shirt costs Rs. 1,500."

Let's analyze it using IPOCE.

#### Input

Number of shirts = 3  
Price per shirt = 1500

#### Process

total = number × price  
total = 3 × 1500

#### Output

Total = Rs. 4500

## **Constraint**

Number of shirts should probably be:

number  $\geq 0$

## **Edge Case**

What if:

number = 0

Then:

total = Rs. 0

So even this simple problem can be analyzed systematically.

---

# **5. The most important idea**

When you receive a programming question, **don't immediately start writing code.**

First ask:

- What is coming in?**
- What should I do with it?**
- What should come out?**
- What restrictions exist?**
- What tricky cases can happen?**

That's the beginning of **problem solving**.

## **Exam definition**

**A problem is a request that can be solved by transforming inputs into outputs under specific rules and constraints.**

## **Remember**

**Problem  $\rightarrow$  IPOCE  $\rightarrow$  Solution**

That's the first foundation of programming.

# Lecture 2 — Introduction to Problem Solving

## Topic 2: How to Read and Understand a Problem

Before writing code, the **first job of a programmer is to understand exactly what the problem is asking.**

A programming question may look like a long paragraph, but we can break it into simple pieces.

Your lecture gives a very useful method called the:

## I-P-O-C-E Checklist

---

### 1. I — Inputs

First ask:

**What exactly comes into the program?**

Look for the information the program needs.

#### Example

"Write a program that calculates the total price of 5 books, where each book costs Rs. 500."

Inputs are:

Number of books = 5

Price of one book = 500

So we have:

Input → 5, 500

---

### 2. P — Process

Now ask:

### **What should we do with those inputs?**

Look for:

- Formulas
- Calculations
- Decisions
- Rules
- Thresholds

For our example:

Total = number of books × price of one book

Therefore:

Total = 5 × 500

= 2500

The multiplication is the **process**.

---

## **3. O — Outputs**

Now ask:

### **What exactly must come out of the program?**

Our program should produce:

Total price = Rs. 2500

So:

Input → 5 books, Rs.500 each

Process → multiply

Output → Rs.2500

---

## **4. C — Constraints**

Now ask:

### **Are there any restrictions or limits?**

For example, the problem might say:

"The number of books must be between 1 and 100."

That's a constraint.

$$1 \leq \text{books} \leq 100$$

You must notice these restrictions before writing your program.

Your lecture specifically mentions **ranges, formats, and limits** as things to look for.

---

## 5. E — Edge Cases

Finally ask:

**What unusual or boundary situations could occur?**

Suppose the problem allows zero books.

What happens if:

$$\text{books} = 0$$

Then:

$$\text{total} = 0 \times 500$$

$$= 0$$

Other possible edge cases could be:

- Empty input
- Zero
- Minimum value
- Maximum value
- Invalid values
- Equal values
- Boundary conditions

Your lecture specifically tells you to consider **boundaries, invalid inputs, and ties** when appropriate.

---

## 6. A Very Useful Reading Trick

Your lecture gives this practical tip:

**Circle nouns and underline verbs.**

Why?

Because **nouns often represent data/variables**, while **verbs often represent actions**.

## Example

"Calculate the average of three **marks** and display the **result**."

Possible nouns:

- marks
- result
- average

Possible action:

- calculate
- display

So you can start converting the English problem into programming concepts.

---

## 7. Fast Pre-Coding Routine

Your lecture also recommends a quick routine before coding:

### Step 1 — Take sample examples

Copy examples from the question and turn them into test cases.

### Step 2 — Build a decision table if necessary

If the problem has many rules or conditions, organize them in a small table.

### Step 3 — Sketch the normal path first

Think about the **happy path** — what normally happens.

Then think about the exceptions.

---

## 8. Let's Practice

Suppose the problem says:

"A student enters his marks. If the marks are 50 or above, display 'Pass'. Otherwise, display 'Fail'. Marks must be between 0 and 100."

Let's analyze it.

### I — Input

Student marks

### P — Process

IF marks  $\geq$  50

    Pass

ELSE

    Fail

### O — Output

Pass

or

Fail

### C — Constraint

$0 \leq \text{marks} \leq 100$

### E — Edge Cases

Important boundaries:

marks = 0

marks = 49

marks = 50

marks = 100

Notice something important:

**50 is an edge/boundary value.**

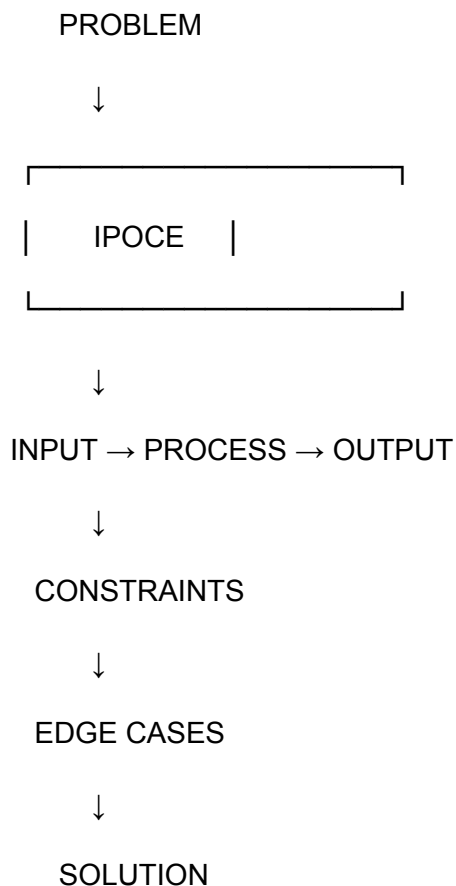
Because that's exactly where the result changes from **Fail** → **Pass**.

---

## ★ The Programmer's Mindset

When you see a long programming question, **don't panic and don't immediately code**.

Break it down:



This is how you turn a **word problem** into **something you can implement**.

### Exam Point

**I-P-O-C-E =**

**Inputs → Process → Outputs → Constraints → Edge Cases**

# Lecture 2 — Introduction to Problem Solving

## Topic 3: Problem-Solving Techniques

Once we understand a problem using **IPOCE**, the next question is:

**How are we going to solve it?**

Your lecture introduces three important ways to represent and analyze a solution:

1. **Algorithm**
2. **Pseudocode**
3. **Flowchart**

The important idea is that these are **three different views of the same logic**.

---

## 1. Algorithm

An **algorithm** is a sequence of steps that explains how to solve a problem.

For example, imagine the problem is:

Find the larger of two numbers.

An algorithm could be:

Step 1: Take two numbers.

Step 2: Compare the two numbers.

Step 3: If the first number is larger, select it.

Step 4: Otherwise, select the second number.

Step 5: Display the larger number.

Notice that we haven't written Python, C++, Java, or any other programming language.

We're simply describing the **logic**.

**Think:**

**Algorithm = What should I do?**

---

## 2. Pseudocode

After creating the algorithm, we can express the same logic in a form that looks more like programming code.

For example:

INPUT A

INPUT B

IF A > B THEN

    PRINT A

ELSE

    PRINT B

END IF

This is **pseudocode**.

It is closer to actual programming than an ordinary algorithm, but it is still not real executable code.

Your lecture describes pseudocode as a **non-formal language** used to represent an algorithm and says that it cannot be directly executed by a computer.

**Think:**

**Pseudocode = How can I describe my logic in a code-like way?**

---

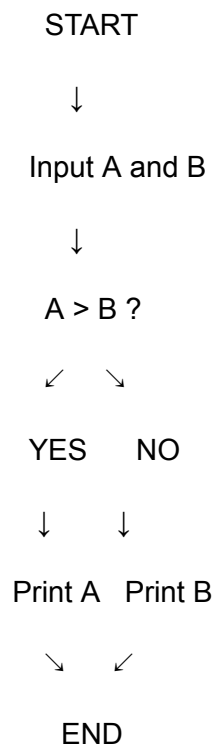
## 3. Flowchart

A **flowchart** represents the same logic **visually**.

Instead of writing every step as text, we use shapes connected by arrows.

Your lecture defines a flowchart as a graphical representation of an algorithm where different shapes represent steps and arrows show their order.

For example:



The diagram makes the **flow of decisions** easy to see.

**Think:**

**Flowchart = How can I see my logic?**

---

## 4. The Same Problem — Three Views

Let's put everything together.

**Problem:**

Find the larger of two numbers.

**Algorithm**

1. Take A and B.
2. Compare A and B.
3. If A is greater than B, output A.
4. Otherwise, output B.

## Pseudocode

INPUT A

INPUT B

IF A > B THEN

    PRINT A

ELSE

    PRINT B

END IF

## Flowchart

START



Input A, B



A > B?



YES    NO



Print A    Print B



END

Same problem.

Same logic.

**Three representations.**

---

## 5. Why Use These Techniques?

Imagine you're given a difficult programming problem.

If you immediately start coding, you might get confused.

Instead:

Problem



Understand it



Algorithm



Pseudocode



Flowchart



Code

This gives you a structured path toward the final program.

Your lecture summarizes their purposes as:

- **Algorithm** → think about the logic
- **Pseudocode** → prepare for coding
- **Flowchart** → visualize and debug the logic

---

## ★ The Key Difference

| Technique | Main Purpose                           |
|-----------|----------------------------------------|
| Algorithm | Describe the solution in ordered steps |

**Pseudocode**     Describe the solution using code-like language

**Flowchart**       Represent the solution visually

**Easy way to remember:**

**Algorithm = Steps**

**Pseudocode = Code-like steps**

**Flowchart = Visual steps**

---

## Exam-ready definition

**Problem-solving techniques** are methods used to analyze and represent the logic of a solution before or during programming. The lecture presents **algorithms, pseudocode, and flowcharts** as three views of the same underlying logic.

# Lecture 2 — Introduction to Problem Solving

## Topic 4: Algorithm

Now we focus **only on Algorithm**.

### 1. What is an Algorithm?

An **algorithm** is a **finite, ordered sequence of unambiguous steps** that transforms inputs into outputs and eventually stops.

In simple words:

**An algorithm is a step-by-step recipe for solving a problem.**

Think about making tea.

You don't simply tell someone:

"Make tea."

You give them steps:

1. Take water.
2. Boil the water.
3. Add tea.
4. Add sugar.
5. Add milk.
6. Serve the tea.

Those steps form an **algorithm**.

---

## 2. Why Does an Algorithm Matter?

Before writing actual programming code, we need to know:

**What exactly should the computer do?**

An algorithm answers this question.

For example, suppose we want to calculate the area of a rectangle.

Instead of immediately writing Python, first create the algorithm:

Step 1: Get the length.

Step 2: Get the width.

Step 3: Multiply length by width.

Step 4: Display the result.

Step 5: Stop.

Only after understanding these steps do we convert them into code.

---

## 3. Properties of a Good Algorithm

Your lecture gives five core properties, remembered as:

**F-D-I-O-E**

## F — Finiteness

The algorithm must **end**.

It cannot continue forever.

✗ Bad:

Step 1: Print Hello

Step 2: Print Hello

Step 3: Print Hello

...

Forever

A proper algorithm eventually reaches:

END

---

## D — Definiteness

Every step must be **clear and unambiguous**.

For example:

✗ Poor instruction:

"Add some sugar."

How much?

Better:

"Add 2 teaspoons of sugar."

The computer needs precise instructions.

---

## I — Input

An algorithm can have **zero or more well-defined inputs**.

Example:

Input: length

Input: width

For some algorithms, there may be no external input.

---

## O — Output

An algorithm should produce **at least one meaningful result**.

Example:

Input:

length = 10

width = 5

Output:

area = 50

---

## E — Effectiveness

Every step should be **simple enough to actually perform**.

For example:

Multiply length by width.

is an effective instruction.

---

# 4. Complete Example

## Problem

Find the total price of 3 products when each product costs Rs. 500.

## Algorithm

Step 1: Start.

Step 2: Input quantity.

Step 3: Input price.

Step 4: Multiply quantity by price.

Step 5: Display total price.

Step 6: End.

With actual values:

quantity = 3

price = 500

total = 3 × 500

= 1500

Output:

Total price = Rs. 1500

---

## 5. Algorithm vs Code

This distinction is **very important**.

### Algorithm

1. Input two numbers.
2. Add the numbers.
3. Display the result.

### Python code

```
a = int(input())
```

```
b = int(input())
```

```
result = a + b
```

```
print(result)
```

The algorithm is **language-independent**.

The code is written in a **specific programming language**.

So:

**Algorithm = logic**

**Code = implementation of that logic in a programming language**

---

## ★ Remember F-D-I-O-E

| Letter | Property      | Meaning                 |
|--------|---------------|-------------------------|
| F      | Finiteness    | Must eventually end     |
| D      | Definiteness  | Steps must be clear     |
| I      | Input         | Well-defined inputs     |
| O      | Output        | Meaningful result       |
| E      | Effectiveness | Steps must be practical |

### Exam definition

**An algorithm is a finite, ordered sequence of unambiguous steps that transforms inputs into outputs and always stops.**

# Lecture 2 — Introduction to Problem Solving

## Topic 5: Pseudocode

Now we focus **only on Pseudocode**.

---

### 1. What is Pseudocode?

**Pseudocode** is a way of representing an algorithm using **simple, code-like language**.

Your lecture describes it as a **non-formal language** that helps programmers write algorithms. It is designed for humans to read and **cannot be executed directly by a computer**.

The word **pseudo** means "**not real**."

So:

**Pseudocode = "not real code."**

It looks somewhat like programming code, but it is mainly used for **planning and communicating logic**.

---

### 2. Why Do We Use Pseudocode?

Suppose you have an algorithm:

1. Get two numbers.
2. Add them.
3. Display the result.

Before writing Python, you can convert it into pseudocode:

INPUT A

INPUT B

SET result  $\leftarrow$  A + B

PRINT result

Now the logic is much closer to actual programming.

So pseudocode acts as a **bridge between an algorithm and real code**.

Problem



Algorithm



Pseudocode



Actual Code

---

### 3. Pseudocode Is Not a Programming Language

This is very important.

Pseudocode:

- Doesn't have one fixed syntax.
- Isn't directly executed.
- Is written for humans.
- Can resemble different programming languages.

Your lecture specifically states:

**No specific standard for writing pseudocode exists.**

For example, both of these can communicate the same idea:

IF marks >= 50 THEN

    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF

or:

IF marks  $\geq$  50

    OUTPUT "Pass"

OTHERWISE

    OUTPUT "Fail"

The exact syntax can vary, but the **logic** is what matters.

---

## 4. Common Pseudocode Keywords

Your lecture recommends clear keywords such as:

### INPUT

Used to receive information.

INPUT age

Meaning:

    Get the user's age.

---

### SET

Used to assign a value.

SET total  $\leftarrow$  price  $\times$  quantity

Meaning:

    Calculate price  $\times$  quantity and store it in total.

---

### IF / ELSE

Used for decisions.

IF marks  $\geq$  50 THEN

```
    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF
```

---

## **ELSE IF**

Used when there are multiple conditions.

```
IF marks >= 80 THEN

    PRINT "A"

ELSE IF marks >= 70 THEN

    PRINT "B"

ELSE

    PRINT "C"

END IF
```

---

## **WHILE**

Used for repetition while a condition remains true.

```
WHILE number < 5

    PRINT number

    SET number ← number + 1

END WHILE
```

---

## **FOR**

Used for repetition over a range or collection.

```
FOR i FROM 1 TO 5

    PRINT i
```

END FOR

---

## **PRINT / RETURN**

Used to produce or return a result.

PRINT total

or:

RETURN total

---

## **5. Important Rule: One Action Per Line**

Your lecture recommends:

**One action per line and indent blocks.**

For example:

INPUT price

INPUT quantity

SET total  $\leftarrow$  price  $\times$  quantity

PRINT total

This is much easier to understand than putting everything into one long sentence.

---

## **6. Example: Calculate Rectangle Area**

### **Problem**

Calculate the area of a rectangle.

Formula:

Area = Length  $\times$  Width

### **Pseudocode**

INPUT length

INPUT width

SET area  $\leftarrow$  length  $\times$  width

PRINT area

That's it.

Notice that we haven't written Python yet.

---

## 7. Example With a Decision

### Problem

Determine whether a student passed or failed. Passing marks are 50.

### Pseudocode

INPUT marks

IF marks  $\geq$  50 THEN

    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF

This clearly represents the decision.

---

## 8. Algorithm vs Pseudocode

This distinction is important for exams.

## Algorithm

1. Take marks.
2. Check whether marks are at least 50.
3. If yes, display Pass.
4. Otherwise, display Fail.

## Pseudocode

INPUT marks

IF marks  $\geq$  50 THEN

    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF

Both describe the **same logic**.

The difference is:

**Algorithm** → ordinary structured steps

**Pseudocode** → code-like representation of those steps

---

## Remember

**Pseudocode is:**

**Human-readable + code-like + language-independent + not directly executable**

And the key idea from your lecture is:

**Pseudocode represents an algorithm and helps programmers prepare for actual coding.**

**Quick memory trick**

**Algorithm = THINK**

**Pseudocode = PLAN**

**Code = BUILD**

# Lecture 2 — Introduction to Problem Solving

## Topic 6: Flowchart

Now we focus **only on Flowcharts**.

---

### 1. What is a Flowchart?

A **flowchart** is a **graphical representation of an algorithm**.

Instead of explaining the solution only with words, we use **different shapes connected by arrows** to visually show how the solution works.

In simple words:

**Flowchart = A visual map of an algorithm.**

For example, instead of writing:

Take marks

↓

Check marks

↓

If marks  $\geq 50 \rightarrow$  Pass

Otherwise  $\rightarrow$  Fail

we can represent the same logic using flowchart symbols.

---

## 2. Why Do We Use Flowcharts?

A flowchart helps us **see the flow of a program**.

It is especially useful when a problem contains:

- Decisions
- Multiple paths
- Repeated steps
- Complicated logic

Your lecture describes flowcharts as a way to **visualize the path** of an algorithm.

For example:

START



Input marks



marks  $\geq$  50?



YES

NO



PASS

FAIL



END

You can immediately see that there are **two possible paths**.

---

## 3. Basic Flowchart Idea

A flowchart consists of:

## Shapes

Different shapes represent different types of operations.

## Arrows

Arrows show the **direction/order of execution**.

Your lecture specifically describes flowcharts as diagrams made from **boxes, diamonds, and other shapes connected by arrows**.

Think of it like following a road:

START

↓

Step 1

↓

Step 2

↓

Decision?

↙   ↘

Path A   Path B

The arrows tell you **where to go next**.

---

## 4. Example: Passing or Failing

Let's take the problem:

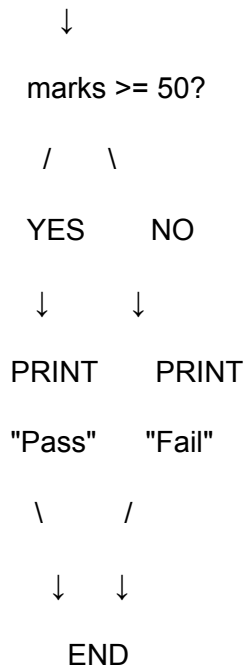
If marks are 50 or above, display "Pass"; otherwise display "Fail".

### Flowchart logic

START

↓

INPUT marks



The **diamond** represents the decision:

marks >= 50?

There are two possible paths:

YES → Pass

NO → Fail

---

## 5. Flowchart vs Pseudocode

They represent the **same underlying logic**, but in different forms.

### Pseudocode

INPUT marks

IF marks >= 50 THEN

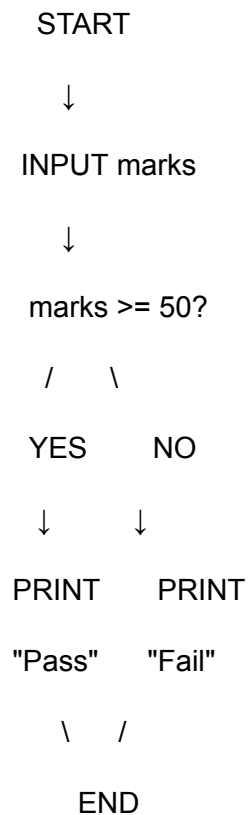
    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF

## Flowchart



So:

**Pseudocode uses words and code-like structure.**

**Flowchart uses shapes and arrows.**

---

## 6. Flowchart in the Problem-Solving Process

The lecture gives us this overall progression:

Problem



Understand the problem



Identify IPOCE



Create Algorithm



Write Pseudocode



Draw Flowchart



Test / Dry Run



Code

The flowchart is therefore a **planning and visualization tool before actual coding**.

---

## 7. Why Is Visualization Powerful?

Suppose your program has many conditions.

Written logic can become difficult to follow:

IF A...

IF B...

...

ELSE...

ELSE...

A flowchart can make the different paths much easier to see.

It can help you find:

- Missing conditions
- Incorrect paths
- Logic mistakes
- Unnecessary steps

- Where the program should stop

That's why your lecture describes the flowchart as a useful way to **visualize and debug logic**.

---

## ★ Most Important Concept

Remember these three:

**Algorithm**

**Steps**

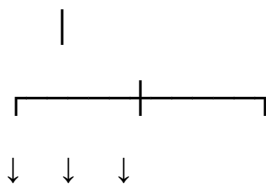
**Pseudocode**

**Code-like steps**

**Flowchart**

**Visual representation of those steps**

SAME LOGIC



Algorithm Pseudo Flowchart

They are **not three different solutions**.

They are **three different ways of representing the same solution logic**.

**Exam Definition**

**A flowchart is a graphical representation of an algorithm in which shapes represent steps and arrows show their sequence or flow.**

# Lecture 2 — Introduction to Problem Solving

## Topic 7: From Words → Plan

Now we focus **only on Topic 7: From Words → Plan**.

A programming problem is usually written in **normal human language**. Before we can write code, we need to transform those words into a clear programming plan.

Your lecture gives a practical sequence for doing this.

---

### 1. Start with the Problem

Suppose the question says:

"Write a program that takes a student's marks and displays whether the student passed or failed."

This is written in ordinary English.

Our job is to convert it into something a computer program can follow.

---

### 2. Identify the Variables and Their Types

First, identify the information involved.

Here we have:

marks

Then determine its type.

For example:

marks → integer

Your lecture recommends listing variables along with their types, such as:

- `int`
- `double`

- `boolean`
- `String`

So we might write:

Variable: marks

Type: int

---

### 3. Design the Rules

Next, determine **what rules the program must follow**.

Our problem says:

If marks are 50 or above → Pass  
Otherwise → Fail

So our rule becomes:

IF marks  $\geq$  50

Pass

ELSE

Fail

The important thing is to put the rules in the correct order.

Your lecture describes this as designing the rules as **ordered checks**.

---

### 4. Choose the Control Structure

Now ask:

**What programming structure do I need?**

There are three major possibilities mentioned in your lecture:

#### **Sequence**

Steps happen one after another.

Input



Calculate



Display

## Selection

The program chooses between alternatives.

Condition?



YES    NO

Usually:

if / else

or:

switch

## Repetition

The program repeats something.

For example:

Repeat 10 times

This uses **loops**.

Your lecture identifies these as:

- **Sequence**
- **Selection**
- **Repetition**

---

# 5. Draft the Algorithm

Now convert your understanding into a sequence of steps.

For our problem:

Step 1: Start.

Step 2: Input marks.

Step 3: Check whether marks  $\geq 50$ .

Step 4: If true, display "Pass".

Step 5: Otherwise, display "Fail".

Step 6: End.

Now we have a clear algorithm.

---

## 6. Draft the Pseudocode

Next, express the same logic in code-like form:

INPUT marks

IF marks  $\geq 50$  THEN

    PRINT "Pass"

ELSE

    PRINT "Fail"

END IF

Now the solution is almost ready to become real code.

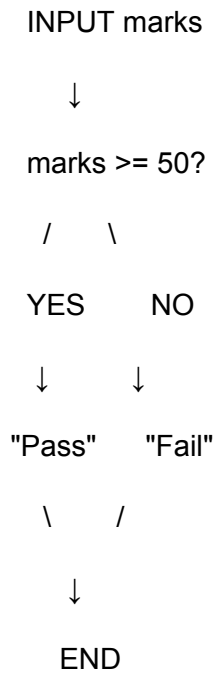
---

## 7. Draw a Flowchart

We can visualize the same logic:

START

↓



The flowchart lets us **see the path** that the program will follow.

Your lecture describes this as visualizing the path and notes that flowcharts can be powerful even when optional.

---

## 8. Dry Run the Solution

Before writing actual code, **test the plan manually**.

Your lecture calls this a **dry run** using sample cases.

Let's test:

### Test 1

marks = 80

Check:

80 >= 50 → TRUE

Output:

Pass

### Test 2

marks = 35

Check:

$35 \geq 50 \rightarrow \text{FALSE}$

Output:

Fail

The logic works for these examples.

---

## Complete Process

So when you receive a programming problem written in words:

WORD PROBLEM



Identify variables



Identify their types



Design the rules



Choose control structures



Algorithm



Pseudocode



Flowchart



Dry Run



CODE

This is the important idea behind **turning words into a programming plan**.

## Exam Point

The lecture's process is essentially:

**Identify variables → design rules → choose control structures → draft algorithm/pseudocode → visualize with a flowchart → dry-run with sample cases.**

# Lecture 2 — Introduction to Problem Solving

## Topic 8: Real-World Problem — Making Tea ☕

Now we apply everything we've learned to a **real-world problem**: making tea.

Your lecture uses making tea as a basic example of how we can take a real-life situation and turn it into a structured problem-solving process.

---

## 1. The Problem

Imagine your friend messages:

"I'm on my way — 5 minutes!"

You need to make a cup of tea.

At first, "**make tea**" sounds simple.

But from a computer-science perspective, we need to break it into:

**Input** → **Process** → **Output** → **Constraints** → **Edge Cases**

---

## 2. I — Input

What do we need?

According to the lecture:

Water

Tea

Milk

Sugar

More specifically, we can represent quantities as:

waterMl

teaTsp

sugarTsp

milkMl

These are the **inputs** to our problem.

---

## 3. P — Process

What do we do with those inputs?

The lecture gives this basic process:

Boil water

↓

Add tea

↓

Add sugar



Add milk

So our process is essentially:

**Boil** → **Tea** → **Sugar** → **Milk**

---

## 4. O — Output

What should we get at the end?

A cup of tea

That's our **output**.

---

## 5. C — Constraints

Now consider the restrictions.

The lecture gives examples such as:

- Boiling time
- Ingredients must exist
- Keep the units consistent, such as **ml** and **tsp**

For example, we shouldn't accidentally treat:

200 ml

as though it were:

200 tsp

The units matter.

---

## 6. E — Edge Cases

Now think about unusual situations.

The lecture gives:

Tea = 0

Milk = 0

Sugar = 0

For example:

**Sugar = 0**

The tea can still be prepared, but it will be **without sugar**.

**Milk = 0**

The process needs to account for having no milk.

**Tea = 0**

Now we have a much bigger problem: we cannot really produce normal tea without tea.

This is why **edge cases matter**.

---

## 7. Turn It Into an Algorithm

Now we convert our understanding into ordered steps.

The lecture's algorithm is essentially:

Step 1: Turn on the stove.

Step 2: Take waterML, teaTsp, sugarTsp, milkML.

Step 3: Boil water.

Step 4: Add tea.

Step 5: Add sugar.

Step 6: Add milk.

Step 7: Pour the tea into the cup.

Step 8: Turn off the stove.

Step 9: End.

The lecture also represents the total liquid approximately as:

$\text{totalMI} \leftarrow \text{waterMI} + \text{milkMI}$

Notice what we have done:

**Real-world activity** → **structured algorithm**

---

## 8. Convert the Algorithm Into Pseudocode

Now we make it more code-like.

The lecture gives:

INPUT waterMI, teaTsp, sugarTsp, milkMI

OUTPUT "🔥 Boil ", waterMI

OUTPUT "🍵 Add ", teaTsp

OUTPUT "🥄 Add ", sugarTsp

OUTPUT "💕 Add ", milkMI

$\text{totalMI} \leftarrow \text{waterMI} + \text{milkMI}$

OUTPUT "☕ Tea ready! ≈ ", totalMI

END

This is **not Python code**.

It is pseudocode showing the logic in a code-like format.

---

## 9. What Did We Learn From the Tea Problem?

This example is important because it demonstrates the complete thinking process:

Real-world problem



Identify Inputs



Identify Process



Identify Output



Identify Constraints



Identify Edge Cases



Create Algorithm



Create Pseudocode



Create Flowchart

The point isn't really **making tea**.

The point is learning how to take a **real-world task** and turn it into a **structured computational solution**.

---

## ★ Exam Point

The tea example demonstrates how the **IPOCE method** can be applied to a real-world problem:

**Input:** water, tea, sugar, milk

**Process:** boil → add tea → add sugar → add milk

**Output:** cup of tea

**Constraints:** boiling time, ingredients, consistent units

**Edge cases:** zero tea, zero milk, zero sugar

# Lecture 2 — Introduction to Problem Solving

## Topic 9: Practice Problem — Make Tea: Algorithm

We already identified the **IPOCE** of the tea problem. Now the lecture takes the next step: turning that understanding into a **finite algorithm**.

### The problem

Your friend is coming in 5 minutes and wants tea.

We need to create a sequence of steps that tells us exactly what to do.

---

## 1. The Algorithm

The lecture gives this sequence:

Step 1: Turn on the stove.

Step 2: Take:

waterMl

teaTsp

sugarTsp

milkMl

Step 3: Boil water.

Add tea.

Add sugar.

Add milk.

Step 4: Pour the tea into the cup.

Step 5: Turn off the stove.

Step 6: End.

---

## 2. Why is this an Algorithm?

Look carefully at what we have.

### **It has an order**

We don't randomly perform the actions.

Turn on stove



Take ingredients



Boil



Add ingredients



Pour



Turn off stove



END

Each step has a position in the sequence.

## **It has an ending**

The final step is:

END

So the process doesn't continue forever.

## **It has clear actions**

For example:

Boil water

Add tea

Add sugar

Add milk

These are specific actions rather than vague instructions.

---

# **3. Algorithm vs Real-Life Instructions**

This is the important computer-science idea.

A human might simply say:

"Make tea."

But that's not enough for a computer.

We need to break the task down:

Make tea

↓

Turn on stove

↓

Get ingredients

↓

Boil water

↓

Add tea

↓

Add sugar

↓

Add milk

↓

Pour into cup

↓

Turn off stove

**Problem → smaller steps → algorithm**

This is how programmers convert a real-world task into something that can eventually be implemented in a program.

---

## 4. The Algorithm Can Then Become Pseudocode

Once we have the algorithm, we can represent it in pseudocode.

The lecture gives:

INPUT waterMI, teaTsp, sugarTsp, milkMI

OUTPUT "🔥 Boil ", waterMI

OUTPUT "🍵 Add ", teaTsp

OUTPUT "🥄 Add ", sugarTsp

OUTPUT "💧 Add ", milkMI

totalMI ← waterMI + milkMI

OUTPUT "☕ Tea ready! ≈ ", totalMI

END

Notice the progression:

Real-world problem



IPOCE



Algorithm



Pseudocode

That's the main lesson of this example.

---

### ★ Key Point

The **algorithm** is the ordered plan.

The **pseudocode** is a code-like representation of that plan.

The lecture uses the tea example to demonstrate how a simple real-world task can be transformed into structured computational logic.

## Topic 10 — Practice Problem: Make Tea → Pseudocode

Now we move to **only the pseudocode part** of the Tea problem. We will not do the flowchart yet.

The lecture gives this pseudocode:

INPUT waterMI, teaTsp, sugarTsp, milkMI

OUTPUT "🔥 Boil ", waterMI

OUTPUT "🍵 Add ", teaTsp

OUTPUT "🥄 Add ", sugarTsp

OUTPUT "💕 Add ", milkMI

totalMI  $\leftarrow$  waterMI + milkMI

OUTPUT "☕ Tea ready!  $\approx$  ", totalMI

END

## 1. What is this doing?

Remember:

**Algorithm** = describes the solution in ordered steps.

**Pseudocode** = writes those same steps in a **code-like form**.

So here we are taking our Tea algorithm and expressing it in pseudocode.

---

## 2. INPUT

INPUT waterMI, teaTsp, sugarTsp, milkMI

This means the pseudocode needs four pieces of information:

- **waterMI**  $\rightarrow$  amount of water
- **teaTsp**  $\rightarrow$  amount of tea
- **sugarTsp**  $\rightarrow$  amount of sugar
- **milkMI**  $\rightarrow$  amount of milk

For example:

waterMI = 500

teaTsp = 2

sugarTsp = 3

milkMI = 200

---

### 3. OUTPUT

OUTPUT "🔥 Boil ", waterMI

The program's instruction becomes something like:

Boil 500 ml

Then:

OUTPUT "🍵 Add ", teaTsp

Add 2 teaspoons of tea

Then:

OUTPUT "🥄 Add ", sugarTsp

Add 3 teaspoons of sugar

And:

OUTPUT "💧 Add ", milkMI

Add 200 ml of milk

The lecture uses **OUTPUT** to represent these instructions in the pseudocode.

---

### 4. Calculating **totalMI**

This is the most important logic part:

$\text{totalMI} \leftarrow \text{waterMI} + \text{milkMI}$

The symbol:

$\leftarrow$

means we are **assigning the result** to **totalMI**.

If:

waterMI = 500

milkMI = 200

then:

totalMI  $\leftarrow$  500 + 200

therefore:

totalMI = 700

---

## 5. Final output

OUTPUT "☕ Tea ready!  $\approx$  ", totalMI

So the result would be approximately:

☕ Tea ready!  $\approx$  700

The lecture specifically calculates the approximate tea volume as:

**water + milk.**

---

## 6. END

END

This tells us the pseudocode has reached the end of its instructions.

---

## Understand the whole pseudocode

Think of it as:

INPUT

↓

Boil water

↓

Add tea



Add sugar



Add milk



Calculate total



Show result



END

That's exactly what pseudocode is helping us do: **turn our planned solution into a form that is very close to programming, without writing actual Python/C/Java code.**

## Exam definition

**Pseudocode is a non-formal, code-like way of representing an algorithm so that humans can understand the logic before actual programming.**

## Key idea to remember

**Algorithm → Pseudocode → Actual Code**

We first decide **what steps are needed**, then express those steps in **code-like form**, and only later translate them into an actual programming language.

## Topic 11 — Make Tea → Flowchart

Now we move to the **final topic** of this lecture: the **Tea Flowchart**.

The lecture defines a flowchart as a **graphical representation of an algorithm**. It uses shapes such as boxes and diamonds, connected by arrows. Each shape represents a step, while the arrows show the order in which the steps happen.

## 1. Why do we need a flowchart?

We already have:

**Algorithm** → written steps

**Pseudocode** → code-like steps

**Flowchart** → visual representation

So the flowchart lets us **see the path of the solution** rather than reading every step as text.

---

## 2. Tea problem as a flow

Our Tea solution follows this order:

START

↓

INPUT water, tea, sugar, milk

↓

Boil water

↓

Add tea

↓

Add sugar

↓

Add milk

↓

Calculate totalMI

↓

Tea ready

↓

END

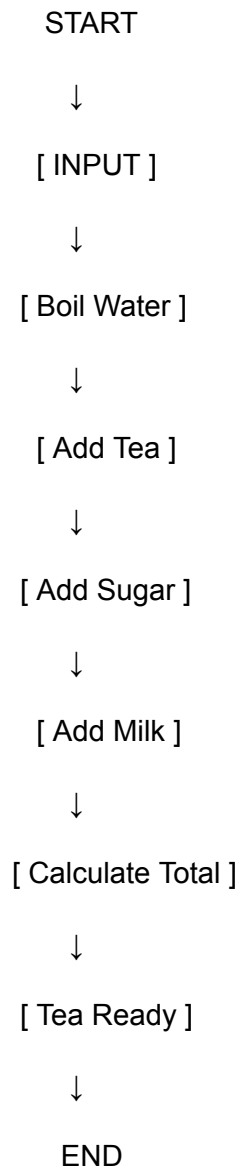
This is the same logic we already developed in the algorithm and pseudocode.

---

### 3. Shapes represent steps

A flowchart converts those instructions into graphical shapes.

Think of it like this:



The **arrows** tell us exactly which step comes next.

---

### 4. The important connection

You should understand this very clearly:

**Problem**



**IPOCE analysis**



**Algorithm**



**Pseudocode**



**Flowchart**



**Actual Program**

The different representations are describing the **same underlying solution logic**.

The lecture specifically presents algorithm, pseudocode, and flowchart as different ways of representing the same logic.



## **Exam-ready definition**

**A flowchart is a graphical representation of an algorithm using shapes and arrows to show the steps and flow of control.**

### **Remember**

**Algorithm = Read the steps**

**Pseudocode = Read code-like steps**

**Flowchart = See the steps**

